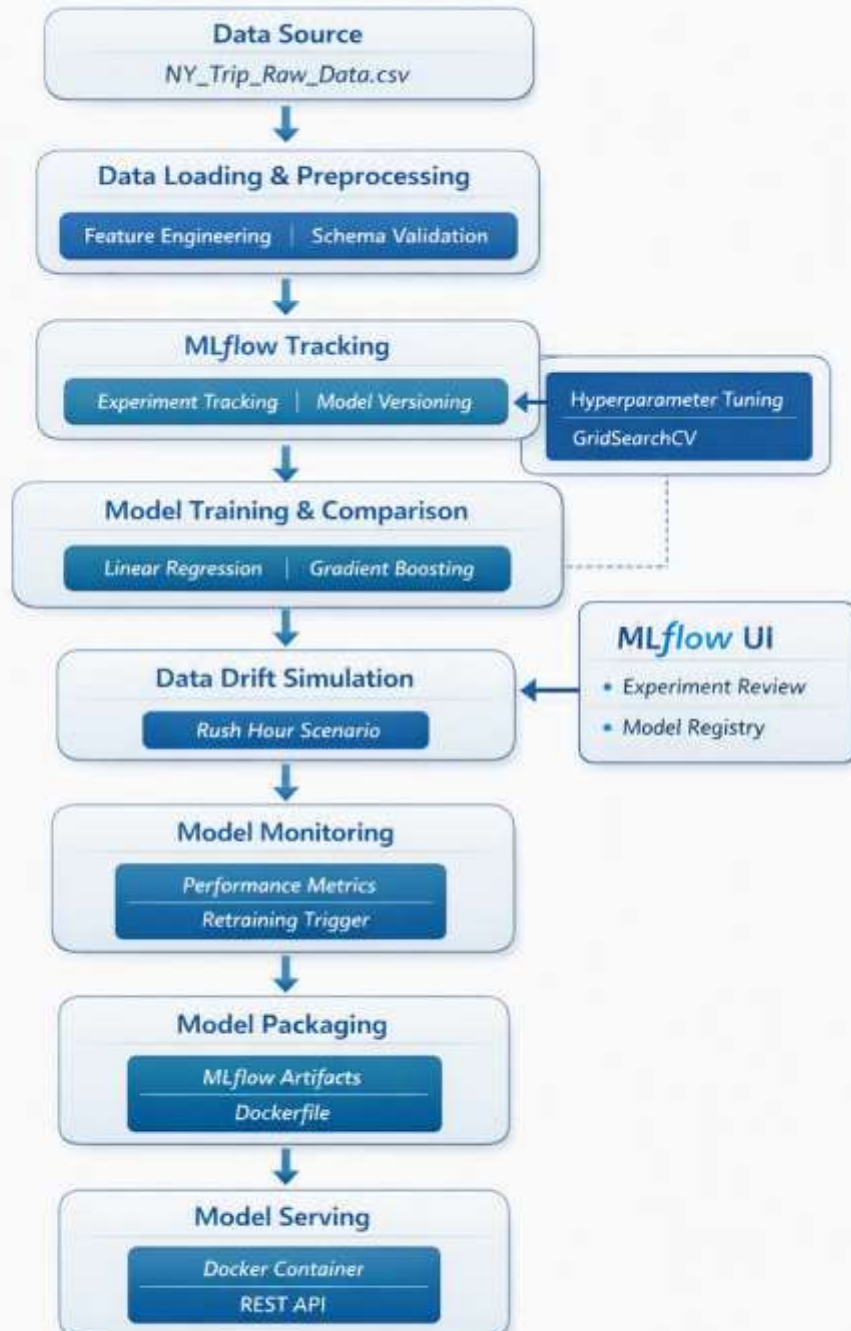


Project Architecture Diagram

MLOps Architecture Pipeline



Description of Components and Flow

This architecture represents an end-to-end MLOps workflow that takes raw taxi trip data through preprocessing, experiment tracking, model training, monitoring, packaging, and deployment as a production-ready prediction service.

1. **Data Source:** The pipeline begins with raw taxi trip data ingested from *NY_Trip_Raw_Data.csv*, which serves as the foundation for training and validating prediction models.
2. **Data Loading and Preprocessing:** The raw dataset is cleaned and prepared through schema validation, datetime conversion, removal of inconsistent records, and feature engineering such as pickup hour, weekend indicator, and trip distance. The resulting processed dataset is versioned for reproducibility.
3. **MLflow Tracking:** MLflow acts as the central experiment-tracking layer, capturing model parameters, evaluation metrics, and artifacts across every training run. This ensures each experiment can be compared, audited, and reproduced.
4. **Model Training and Comparison:** Multiple regression models, including Linear Regression and Gradient Boosting Regressor, are trained and evaluated. Their performance is compared using tracked metrics such as mean squared error and R-squared.
5. **Hyperparameter Tuning:** GridSearchCV is applied to optimize the Gradient Boosting Regressor by testing different parameter combinations and selecting the configuration that delivers the strongest validation performance.
6. **Data Drift Simulation:** A rush-hour surge scenario is introduced to simulate real-world data drift and evaluate how the model behaves when incoming data patterns change.
7. **Model Monitoring:** The trained model is monitored against drifted data using performance thresholds. If degradation exceeds the defined limit, such as a significant drop in R-squared, the workflow signals the need for retraining.
8. **Model Packaging:** The best-performing model is packaged with its required artifacts, dependencies, requirements file, and Dockerfile so it can be deployed consistently across environments.
9. **Model Serving:** The packaged model is exposed as a REST API inside a Docker container, enabling external applications to submit input data and receive predictions.
10. **MLflow UI:** The MLflow interface provides a visual dashboard for reviewing experiment runs, comparing model performance, inspecting artifacts, and managing the model registry.